Alumnos:

Marco Antonio Aguilar Castillo

Cleber Antonio Bolaños Moreno

Osbaldo Alvarez Martinez

Kevin Ricardo Simon Alfaro

Samuel Jonathan Trujillo Bolaños

Docente:

José Miguel Carrera Pacheco

8°B

Proyecto: Conecta-Tehuacán

INVESTIGACION:

S - 13. Protección de rutas y recursos por rol

Protección de rutas y recursos por rol

1. ¿Qué es un middleware?

Un **middleware** es una función que se ejecuta entre la solicitud (request) y la respuesta (response) en el backend. Su objetivo es interceptar la petición antes de que llegue a la lógica principal.

Se utiliza para:

- Validar autenticación
- Verificar permisos
- Procesar datos
- Manejar errores

Ejemplo en Node.js con Express:

```
function verificarToken(req, res, next) {
  const token = req.headers['authorization'];

  if (!token) {
    return res.status(401).json({ mensaje: 'Acceso denegado' });
  }

  // lógica de validación del token
  next();
}
```

2. ¿Cómo se protegen rutas en backend?

Las rutas se protegen utilizando middlewares que validan si el usuario está autenticado antes de permitir el acceso.

Flujo básico:

1. El usuario inicia sesión
2. El servidor genera un token (por ejemplo, JWT)
3. El cliente envía ese token en cada petición
4. El backend valida el token en un middleware

Ejemplo:

```
const express = require('express');
const app = express();

app.get('/perfil', verificarToken, (req, res) => {
  res.json({ mensaje: 'Ruta protegida' });
});
```

También se pueden proteger rutas completas:

```
app.use('/admin', verificarToken, rutasAdmin);
```

3. ¿Qué errores de seguridad son comunes?

Errores frecuentes al proteger rutas:

- No validar el token en todas las rutas protegidas
- Confiar en datos enviados por el cliente (por ejemplo, el rol)
- No manejar expiración de tokens
- Usar secretos débiles en JWT
- No usar HTTPS

- Exponer mensajes de error sensibles
- No sanitizar entradas del usuario
- Permitir acceso sin verificar permisos (solo autenticación)

4. ¿Cómo se valida acceso por rol?

La validación por rol consiste en verificar que el usuario tenga permisos específicos para acceder a ciertos recursos.

Ejemplo de roles:

- admin
- usuario
- invitado

Primero se obtiene el rol desde el token:

```
function verificarRol(rolPermitido) {
  return (req, res, next) => {
    const usuario = req.usuario; // viene del middleware de autenticación

    if (usuario.rol !== rolPermitido) {
      return res.status(403).json({ mensaje: 'Acceso prohibido' });
    }

    next();
  };
}
```

Uso en rutas:

```
app.get('/admin', verificarToken, verificarRol('admin'), (req, res) => {
  res.json({ mensaje: 'Bienvenido administrador' });
});
```

También se puede permitir múltiples roles:

```
function verificarRoles(rolesPermitidos) {
  return (req, res, next) => {
    if (!rolesPermitidos.includes(req.usuario.rol)) {
      return res.status(403).json({ mensaje: 'Acceso denegado' });
    }
    next();
  };
}
```

5. Buenas prácticas

- Separar autenticación y autorización en diferentes middlewares
- No confiar en el frontend para validar permisos
- Usar tokens firmados (JWT) con expiración
- Guardar roles en el backend o dentro del token validado
- Aplicar el principio de menor privilegio
- Registrar accesos y errores (logging)
- Proteger rutas sensibles como /admin o /config

6. Conclusión

La protección de rutas es fundamental para la seguridad de cualquier aplicación. No basta con saber quién es el usuario (autenticación), también es necesario validar qué puede hacer (autorización). El uso correcto de middlewares y validación por roles permite construir sistemas seguros, escalables y confiables.